The PJS Savings Heuristic for the Team Orienteering Problem: Biased Randomisation and Local Search on the Chao Instances

Heuristic Algorithms in Transport and Finance — Master's course assignment
Javier Salvatierra
Jesús Inurria

October 2026

Abstract

The Team Orienteering Problem (TOP) asks for at most m time-limited routes that collect the largest total reward from optional customers. We implement in Rust the savings heuristic of Panadero et al. (PJS), turn it into a biased-randomised multi-start (GRASP), add a 2-opt and insertion local search on elite solutions, and test the three layers on the 353 feasible instances of Chao et al. (about 2 min on a laptop). Over PJS with its best α, GRASP adds 7.98 % to the total reward and the local search brings the gain to 20.26 %. The mean gap to the best-known solutions is 2.04 %, or 2.17 % as Panadero et al. measure it, below their 2.85 % with 128 threads and lower in all seven sets. Trying 21 values of α pays more than iterating longer with one.

1 Introduction

A small fleet of m vehicles leaves an origin and must reach a destination before a deadline. On the way it can visit customers, each with a reward, but there is not enough time to visit them all: the planner decides which customers to serve, which vehicle serves each of them and in which order. This is the Team Orienteering Problem (TOP); it maximises the collected reward instead of minimising distance, and it is NP-hard [1]. Panadero et al. [1] adapt the savings of Clarke and Wright [3] to it, randomise the heuristic and run it on many threads for one second; Li et al. [2] use it for a dynamic variant; we study the static TOP with this heuristic, which the course calls *PJS*.

The test data are the classical instances of Chao et al. [4]: 387 files in seven sets `p1`–`p7`, with 21 to 102 nodes and m between 2 and 4. An instance named `p4.3.k` belongs to set 4 (100 nodes: origin, destination and 98 customers), has $m = 3$ vehicles, and the letter `k` codes the time limit, here $t_{\max} = 50$. Within a set the nodes and rewards are identical; only m and $t_{\max}$ change. In 34 instances no customer fits within $t_{\max}$, so we report on the remaining 353 (the paper speaks of 354). Best-known solutions (BKS) are the column *BEST* of the published results of PSOiA [5]; Panadero et al. use those of Ke et al. quoted in [1], checkable only on the 81 instances of Tables 3–4 of [1] (`p1`–`p2`), where they coincide with ours.

2 Problem formulation and enriched savings

Node 0 is the origin, node $n+1$ the destination and $N' = \{1, \dots, n\}$ the set of customers; customer i has a reward $u_i > 0$, and t_{ij} is the Euclidean travel time between nodes i and j. A route $R = (i_1, \dots, i_L)$ is travelled as $0 \to i_1 \to \cdots \to i_L \to n+1$, takes $t(R) = t_{0i_1} + \sum_{k<L} t_{i_k i_{k+1}} + t_{i_L,n+1}$ and collects $u(R) = \sum_{i \in R} u_i$. The TOP reads

$$\max \sum_{r=1}^{p} u(R_r) \qquad \text{s.t.} \qquad R_r \subseteq N', \quad R_r \cap R_s = \emptyset \ (r \neq s), \qquad p \leq m, \qquad t(R_r) \leq t_{\max} \quad (r = 1, \dots, p). \tag{1}$$

Read literally: at most m routes, no customer served twice, every route within $t_{\max}$, and customers left out cost nothing.

Joining a route that ends at i with a route that starts at j replaces the legs $i \to n+1$ and $0 \to j$ by the direct leg $i \to j$, which saves the time $s_{ij} = t_{i,n+1} + t_{0j} - t_{ij}$. Unlike in the vehicle routing problem, this saving depends on the direction and can be negative, because origin and destination differ in `p1`–`p6`: in `p4.3.k`, 12.8 % of the 9506 ordered pairs have $s_{ij} < 0$ and s_{ij} ranges from -19.74 to 42.57, whereas in `p7`, where both depots coincide, the triangle inequality gives $s_{ij} \geq 0$. Panadero et al. compute one saving per edge, arguing that, since travel times are symmetric, the saving does not depend on the direction [1]; with distinct depots the times are still symmetric but the savings are not, so we keep both directions, as Li et al. do [2]. Time alone ignores the objective, so PJS ranks the pairs by the *enriched saving*

$$s'_{ij} = \alpha\, s_{ij} + (1 - \alpha)\,(u_i + u_j), \qquad \alpha \in [0, 1], \tag{2}$$

which with $\alpha = 1$ counts only time and with $\alpha = 0$ only reward (the paper takes $\alpha \in (0, 1)$; our grid includes both ends). Both terms have the same order of magnitude: in `p4.3.k` rewards range from 1 to 29, so $u_i + u_j$ lies between 2 and 58.

3 PJS, GRASP and local search

3.1 The PJS heuristic

Following Section 4 of [1]:

1. **Dummy solution.** One route $0 \to i \to n+1$ per customer. A customer whose dummy route exceeds $t_{\max}$ is discarded.
2. **Savings list.** The enriched savings (2) of all ordered pairs are sorted in decreasing order.
3. **Merging.** Walking down the list, the arc (i, j) merges the route that ends at i with the route that starts at j if the result fits in $t_{\max}$; the arc is then deleted, used or not.
4. **Selection.** Routes are sorted by reward and the best m are kept.

Table 1: Mean gap to the BKS [5] per set. *PJS*: best of the 21 α; *GRASP*: best without local search; *Final*: elite + local search (10 iterations); *Panadero 128*: Figure 4 of [1] (BKS of Ke et al., dividing by its own reward as in its Tables 3–4), *All* weighted by feasible instances; with that denominator *Final* is 0.60, 0.00, 0.70, 6.38, 1.35, 1.22, 3.61 (*All* 2.17). *Gain*: total reward of Final over PJS.

Set	Nodes	Feasible instances	PJS gap (%)	GRASP gap (%)	Final gap (%)	Panadero 128 gap (%)	BKS reached (Final)	Gain (%)
p1	32	48	6.88	3.15	0.58	0.69	40	8.70
p2	21	33	1.81	0.31	0.00	0.11	33	2.09
p3	33	60	6.84	2.96	0.67	1.44	48	8.40
p4	100	56	29.25	23.73	5.87	7.97	5	38.75
p5	66	74	11.82	4.49	1.30	1.46	35	13.61
p6	64	24	14.45	8.51	1.17	1.42	14	17.44
p7	102	58	19.75	14.48	3.42	5.10	14	28.07
All		**353**	**13.61**	**8.63**	**2.04**	**2.85**	**189**	**20.26**

Table 2: The 81 instances of p1–p2, BKS from Tables 3–4 of [1]; all gaps, those of Panadero et al. included, divide by the BKS. *vs. P-128*: instances better / equal / worse than Panadero et al. with 128 threads; in bold, our best variant.

Method	Mean gap (%)	BKS reached	vs. P-128
PJS, best α	4.82	44	0 / 45 / 36
GRASP, 10 iterations, 21 α	1.99	53	1 / 56 / 24
Panadero et al., 1 thread, 1 s	1.73	52	—
GRASP, 1 s per run, 21 α	0.71	65	4 / 67 / 10
Panadero et al., 128 threads, 1 s	0.44	69	—
GRASP, 10 iterations, 21 α + local search	0.34	73	10 / 65 / 6
GRASP, 100 iterations, 21 α + local search	0.29	74	12 / 62 / 7
GRASP, 1 s per run, 21 α + local search	**0.15**	**77**	**12 / 65 / 4**

The paper gives no value of α, only the rule that it should be close to 0 with heterogeneous rewards and close to 1 with homogeneous ones [1]. The rule does not hold here. Rewards are fixed per set, so there are only seven values of their coefficient of variation (CV), and over the 285 instances in which α matters the correlation between CV and the best α (the mean of tied optima) is $+0.19$ ($+0.43$ with the smallest optimum), with the wrong sign either way; in p4, the most heterogeneous set (CV 0.64), $\alpha = 0$ reaches on average only 68 % of the best reward over its 54 such instances. In preliminary trials, not part of the published code, rules based on the CV or on instance features did not beat a fixed α, because the reward jumps irregularly with α: a small change reorders the savings list and the greedy merging builds different routes. We therefore run the 21 values $\alpha = 0, 0.05, \ldots, 1$ and keep the best.

3.2 GRASP: biased-randomised multi-start

An *iteration* is a complete PJS construction in which the next arc is drawn at random, biased towards the top of the list. A *run* starts from the deterministic solution, performs iterations in series and keeps the best, as one thread of Algorithm 1 of [1] or the multi-start of Algorithm 1 of [2]. We test two biased rules:

- **Geometric** (the paper's): take the k-th remaining arc, $k \sim \mathrm{Geom}(\beta)$ with $\beta = 0.3$ [1]. Only the position matters.
- **Softmax** (our proposal): take each remaining arc with probability proportional to $\exp(s'_{ij}/\tau)$, $\tau = 1$. Arcs with similar savings get similar probabilities.

3.3 Local search on elite solutions

The local search runs at the end, not inside the iterations, on the *elite* of each instance: the deterministic solutions of the three best α and, for every α and rule, the best solutions of the three best runs (ties included); solutions that visit the same customers are kept once. Each elite solution goes through a loop of two moves: (i) **2-opt** inside every route, first improvement; (ii) **insertion** of the unvisited customer with the largest reward that fits somewhere (ties: least added time) at its cheapest position, repeated while a customer is inserted. No move loses reward, so no acceptance criterion is needed; the time freed by 2-opt feeds the next insertion.

4 Implementation

The assignment asks for a Python program; we wrote the code in Rust (rustc 1.99, crate `pjs` in `src/rust`) because the experiment consists of 170 667 independent tasks, which the library `rayon` spreads over all threads. The main modules are `heuristic.rs` (PJS), `grasp.rs`, `elite.rs` and `local_search.rs`. The savings list is sorted with a *stable* sort, so tied arcs keep their generation order; ties in the selection step favour the shorter route; and $t_{\max}$ has a tolerance of 10^{-9}.

Protocol. For every instance and every α: one deterministic construction and 10 runs of 10 iterations with each rule, i.e. 21 tasks per α (441 per instance) and 4200 random constructions per instance. Each run's seed derives from the base seed 1, the instance, α, the rule and the run number, so results are reproducible bit for bit. On the 81 instances of Tables 3–4 of [1] we also ran, over the 21 α, 100 iterations and 1 s of wall-clock time per run (the paper's budget; not exactly reproducible), and 20 runs of 1 s with the best deterministic α only. The quality measure is gap $= 100\,(\mathrm{BKS} - \mathrm{reward})/\mathrm{BKS}$, averaged over instances; the paper divides by its own reward instead, which gives slightly larger gaps (our final solutions: 2.17 % instead of 2.04 %). All runs use 12 threads on an AMD Ryzen 5 5500U laptop (6 cores) under Windows 11; Panadero et al. use Java on a 16-core Xeon (1–128 threads).

Validation. All solutions pass a feasibility check; a separate binary sharing no code re-verifies the 1000 final routes.

5 Computational results

Table 1 gives the mean gap per set and Table 2 the comparison on p1–p2.

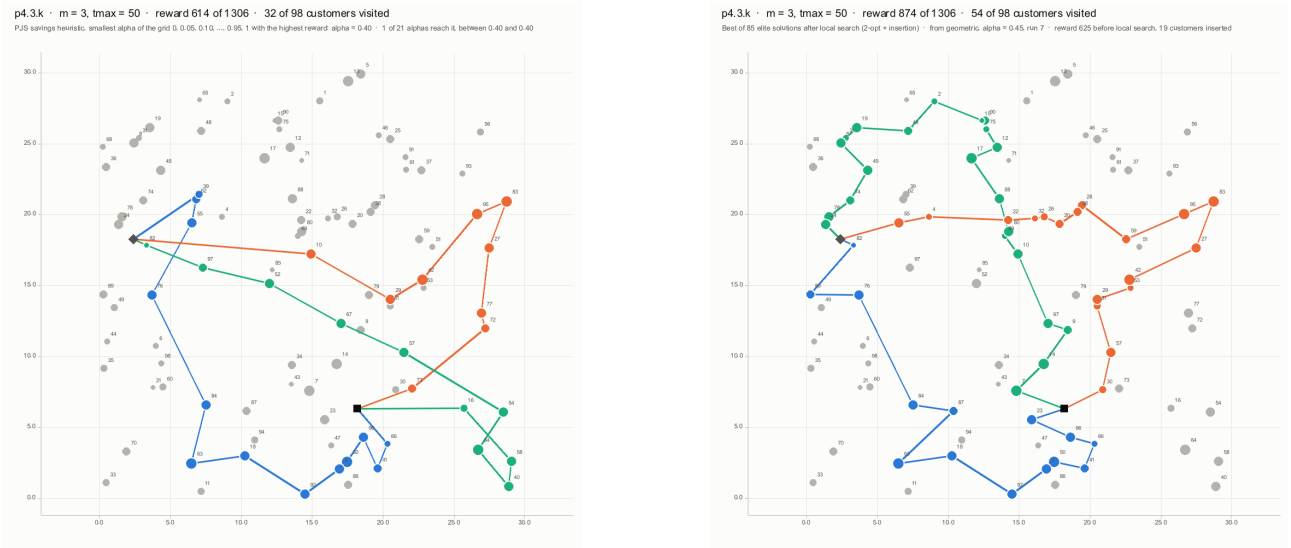


Figure 1: `p4.3.k` ($m = 3$, $t_{\max} = 50$; square: origin, diamond: destination; marker size grows with reward; grey: not visited). Left: PJS with its best $\alpha = 0.40$, reward 614 with 32 customers. Right: final solution, reward 874 with 54 customers, from a geometric run with $\alpha = 0.45$ worth 625 before the local search, which inserted 19 customers.

Each layer adds reward. Over the 353 instances, PJS with its best α collects 157 868; GRASP raises it to 170 469 (+7.98 %), the local search on the deterministic elite alone to 181 103 and on the whole elite to 189 854 (+20.26 %), 3.10 % below the total of the BKS. The local search raises the reward of an elite solution by 18.16 % on average (7.48 customers inserted) and improves 93.6 % of the 18 757 elite solutions. The gain grows with the instance (Table 1): 2–9 % in the small sets `p1`–`p3`, where few customers remain outside, and 38.75 % and 28.07 % in `p4` and `p7`, with about 100 nodes and long routes. Figure 1 shows the mechanism: the PJS routes cross the map along long, almost empty legs, while the final ones sweep dense groups of customers (reward +42.3 %, from 614 to 874). Diversity matters: in 163 of the 353 instances, applying the local search only to the best solution before it would have given less (8 187 reward in total), and on `p1`–`p2` the local search on the deterministic elite alone stops at 2.13 %, against 0.34 % with the GRASP elite.

Comparison with Panadero et al. With 10 iterations per run, the final gap is below that of Panadero et al. with 128 threads in all seven sets (2.17 % vs. 2.85 % with the paper's denominator), most clearly in `p4` and `p7`. On `p1`–`p2` (Table 2) it reaches 0.34 % against 0.44 %, and 0.15 % with 1 s per run (BKS in 77 of 81 instances, worse than the paper in 4). Without the local search no variant reaches the paper's 128 threads, and one of our 1 s runs gives on average 2.45 % (geometric) or 2.33 % (softmax), worse than the 1.73 % of one of its threads. The difference probably lies in what the paper leaves open (tie order, α, one or two savings per edge) and in our runs sharing 6 cores (fewer iterations per run).

Many α vs. many iterations. On `p1`–`p2`, 1 s runs with the best α (on average about 880 000 constructions per instance, leaving out the trivial `p2.4.a`) give a gap of 1.49 %, and 100-iteration runs over the 21 α (42 000 constructions) give 1.45 %: the same quality with about 20 times fewer constructions and a fifth of the thread time (4 s against 20 s per instance). Spreading the budget over α explores more.

Time. The grid takes 127 s on 12 threads for the 387 instances, under 4 s of thread time per instance against 128 s for Panadero et al. with 128 threads (different machines and languages, so only indicative); the local search adds 0.4 s. The 1 s variant spends 420 s of thread time per instance, so its advantage does not come at equal cost.

6 Conclusions

PJS is a greedy pass over a list of enriched savings; even with the best of 21 values of α per instance it stays 13.61 % below the best-known solutions. Adding biased randomisation and a cheap 2-opt and insertion local search on a diverse elite raises the reward by 20 % and brings the mean gap to 2.04 %, below Panadero et al. with 128 threads in all seven sets. The large sets `p4` and `p7` gain most but remain the weak spot (5.87 % and 3.42 %). The next step is a richer neighbourhood (moves between routes, swaps) inside an iterated local search: diversity first, then repair.

Code availability. `cargo build --release` in `src/rust` puts the executable in `ignore/target/release`. From `TOP/`, `pjs grid data/top-data results` with `--png-dir results/figures/routes` and `--ls-png-dir results/figures/routes_ls` reproduces the grid, the local search and Figure 1 (≈ 2 min); `pjs bks` and three runs of `pjs panadero` (see `docs/scratchpad.md`) write the CSV files behind Tables 1–2.

References

[1] J. Panadero, M. Ammouriova, A. A. Juan, A. Agustin, M. Nogal and C. Serrat. Combining parallel computing and biased randomization for solving the team orienteering problem in real-time. *Appl. Sci.*, 11(24):12092, 2021.

[2] Y. Li, M. Peyman, J. Panadero, A. A. Juan and F. Xhafa. IoT analytics and agile optimization for solving dynamic team orienteering problems with mandatory visits. *Mathematics*, 10(6):982, 2022.

[3] G. Clarke and J. W. Wright. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.*, 12(4):568–581, 1964.

[4] I.-M. Chao, B. L. Golden and E. A. Wasil. A fast and effective heuristic for the orienteering problem. *Eur. J. Oper. Res.*, 88:475–489, 1996.

[5] D.-C. Dang, R.-N. Guibadj and A. Moukrim. An effective PSO-inspired algorithm for the team orienteering problem. *Eur. J. Oper. Res.*, 229:332–344, 2013.